Storage class

Decides storage properties of variable (and function)

- Memory
- Life
- Scope
- Default Initial Value

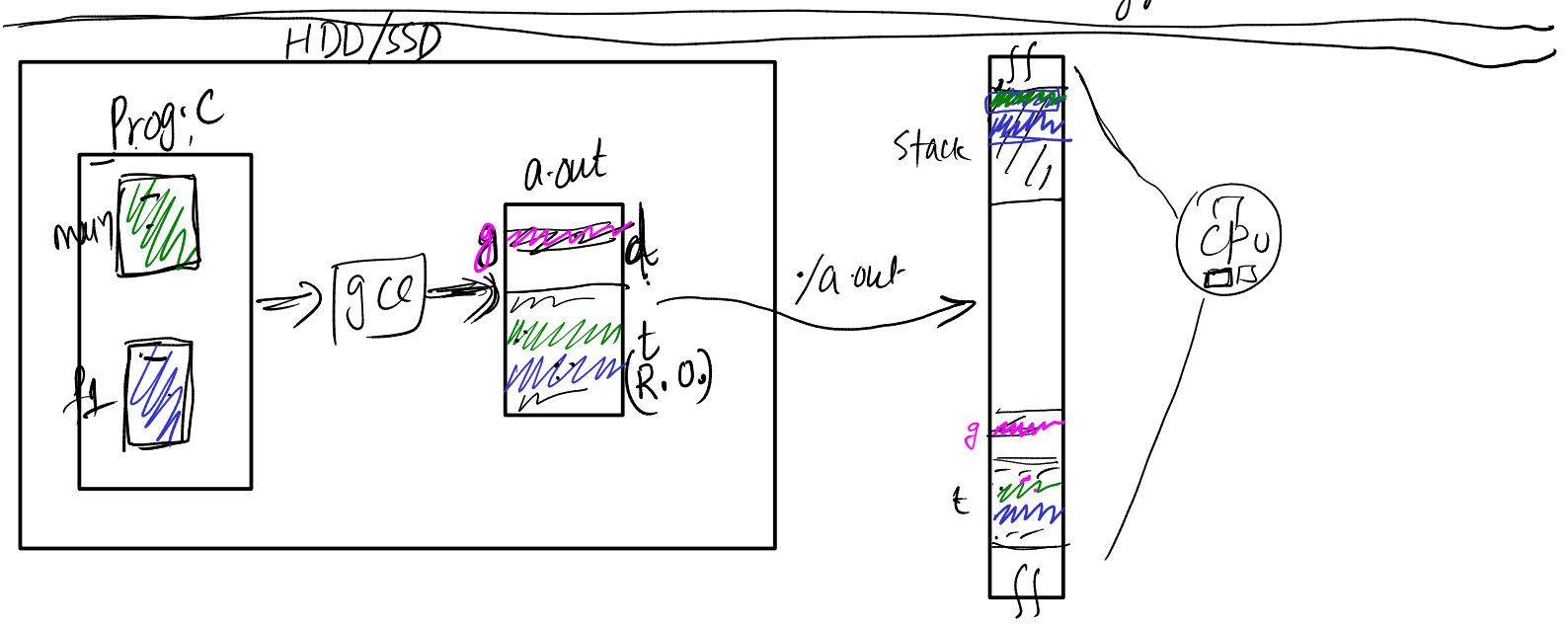
- auto
- register
- static
- extern

Syntax:

<st_class> variable Declaration;

```
static int cnt;        extern float pi;
auto char ch;          register int i;
```

- Recursion
- call-by ref (Pointer)
- st_class
- Debugger



To understand/analyze the content of binary file (like a.out) many tools are present, like nm.

$ nm a.out ↵

In our example program:

- The variable b, got life when f1 is called. — auto
- The " g, got life when prgs loaded into mem. — static

Automatically created, when the function call happens, gets destroyed automatically, with function Return.

Global variable and static variable life NOT depending on function's call/return. They have STATIC life

storage class
- automatic
  - auto
  - register
- static
  - static
  - extern

auto: Memory: stored in stack (seg) frame. Preferrably CPU-Register

register:

Life: same: starts with function call, and gets destroyed with fn Return

(Visibility) Scope: same: Visible/usable inside the block.

D.I.V: same: Undefined

```
int g;
void f1(int);
int main()
{
        int a;
        printf("in main...a=%d g=%d \n",a,g);
        f1(a+g);

}
void f1(int b)
{
printf("in f1..b=%d\n",b);
}
```

---

static / extern

Memory: Data segment

Life: starts with process and gets destroyed with process
↳ App loaded to memory

Scope: Visible to the funct block where declared.
↑ has internal linkage
visible to the entire App (all functions), has external linkage

D.V: Zero by default.

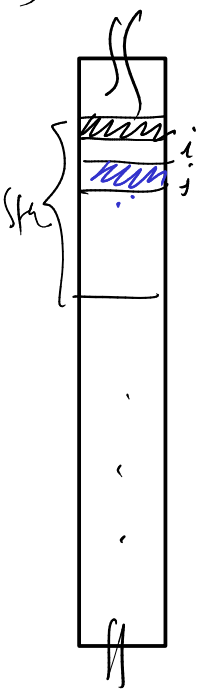
```
main(){ int i;
  register int j;

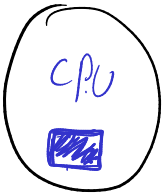
for(i=1; i<=1000000; i++){
  }

for(j=1; j<=1000000; j++){
  }

}
```

```
register int x; //Error
main(){
  printf("%d", x);
}
```





```
#include<stdio.h>
int g;
void f1(int);
void f2(int);
int main()
{
        int a;
        printf("Enter int for a:");
        scanf("%d",&a);
        if(a<0)
             f1(a);
        printf("g=%d\n",g);

}
void f1(int b)
{
        static int s1;
        ++g;
printf("in f1..b=%d\n",b);
printf("Enter new value for b:");
scanf("%d",&b);
if(b>100)
        f2(b);
}

void f2(int c)
{
        static int s2;
        g+=100;
        printf("in f2..c=%d s2=%d\n",c,s2);
}
```